TALLER 1: PILAS DINÁMICAS CON LISTAS ENLAZADAS
Conversión y Evaluación de Expresiones (Notación Polaca)

Objetivo
Implementar un programa informático (en Python) que, mediante una interfaz gráfica y utilizando **pilas dinámicas implementadas con listas enlazadas**, permita:

- Convertir expresiones matemáticas en notación infija a:
  - notación postfija
  - notación prefija
- Evaluar expresiones en notación:
  - postfija
  - prefija

El programa debe implementar las operaciones fundamentales de una pila (push, pop) mediante la **clase Nodo** estudiada, considerando validaciones de entrada, manejo de errores e interfaz gráfica amigable.

Contextualización
Las pilas son estructuras de datos tipo **LIFO (Last In, First Out)**, ampliamente utilizadas en:

- compiladores
- intérpretes
- calculadoras
- procesamiento de expresiones matemáticas

En ciencias de la computación, las expresiones matemáticas pueden representarse en diferentes notaciones:

Tipo	**Ejemplo**
Infija	(A+B)*C
Postfija	AB+C*
Prefija	*+ABC

La conversión y evaluación de estas expresiones requiere el uso de pilas para:

- almacenar operadores
- controlar precedencia
- manejar paréntesis
- resolver operaciones en orden correcto

Por ejemplo:
Expresión infija:
(2+3)*4

Conversión:
Postfija:
23+4*
Evaluación:
2 + 3 = 5
5 * 4 = 20
Resultado final:
20

Actividad para realizar
El estudiante desarrollará un programa con interfaz gráfica que permita:

Parte 1: Conversión de Expresiones
Funcionalidades:

- Ingresar una expresión en notación infija
- Convertir a:
  - notación postfija
  - notación prefija
- Mostrar resultados en pantalla

Parte 2: Evaluación de Expresiones
Funcionalidades:

- Evaluar expresiones en:
  - notación postfija
  - notación prefija
- Mostrar:
  - operaciones realizadas
  - resultado final

Ejemplos
Entrada:
(A+B)*C
Salida Postfija:
AB+C*

Salida Prefija:
*+ABC

Evaluación Postfija
Entrada:
23+4*
Resultado:
20

Evaluación Prefija
Entrada:
*+234
Resultado:
20

Requisitos técnicos
El programa debe implementar obligatoriamente:
Estructura dinámica

- Clase Nodo
- Clase Pila

Operaciones de pila

- push()
- pop()
- Verificación de pila vacía

Algoritmos de Base

- Conversión infijo → postfijo
- Conversión infijo → prefijo
- Evaluación postfija
- Evaluación prefija

Interfaz gráfica

Debe incluir:

- campo de entrada
- botones:
    - convertir a postfijo
    - convertir a prefijo
    - evaluar postfijo
    - evaluar prefijo
    - limpiar
- área de resultados
- validación de expresiones

Consideraciones finales

El programa debe:

- validar expresiones incorrectas
- detectar:
    - operadores inválidos
    - expresiones incompletas
    - paréntesis desbalanceados
- verificar si la pila está vacía antes de realizar operaciones

Pruebas

Se deben realizar pruebas con:

Expresiones válidas

(A+B)*C
(A+B)*(C-D)

Expresiones inválidas

(A+B*
+A

Evaluaciones numéricas

23+4*
*+234

Entrega
El estudiante deberá entregar:

- código fuente completo
- capturas de ejecución
- evidencia de pruebas
- análisis de complejidad

Modalidad
Trabajo en grupos de tres (3) estudiantes.

Rúbrica de Evaluación

Criterio (Peso)	Bajo (0.0 – 2.9)	Básico (3.0 – 3.7)	Alto (3.8 – 4.4)	Superior (4.5 – 5.0)
Funcionalidad del Programa (25%)	El programa no ejecuta correctamente o presenta errores críticos que impiden la conversión y evaluación de expresiones.	El programa ejecuta parcialmente las conversiones o evaluaciones, pero presenta errores frecuentes en los resultados.	El programa convierte y evalúa correctamente la mayoría de expresiones, con pocos errores menores.	El programa convierte y evalúa correctamente expresiones postfijas y prefijas en todos los casos probados, mostrando resultados precisos y consistentes.
Implementación de Pilas Dinámicas (20%)	No utiliza pilas dinámicas o implementa incorrectamente listas enlazadas.	Implementa parcialmente la pila dinámica, pero presenta errores en operaciones como push, pop o peek.	Implementa correctamente las operaciones fundamentales y controla adecuadamente estados vacíos.	Implementa una pila dinámica robusta, modular y eficiente mediante listas enlazadas utilizando correctamente la clase Nodo.
Conversión de Expresiones (15%)	No convierte expresiones o los resultados son incorrectos.	Convierte únicamente expresiones simples y presenta errores de precedencia o manejo de operadores.	Convierte correctamente expresiones con múltiples operadores y paréntesis en la mayoría de casos.	Convierte correctamente expresiones infijas a postfijas y prefijas considerando precedencia, asociatividad y validaciones completas.
Evaluación de Expresiones (15%)	No evalúa expresiones o los resultados son incorrectos.	Evalúa expresiones simples, pero presenta errores frecuentes en operaciones o manejo de operandos.	Evalúa correctamente la mayoría de expresiones postfijas y prefijas.	Evalúa correctamente expresiones complejas mostrando de forma clara las operaciones realizadas y los resultados finales.

Estructura y Organización del Código (10%)	Código desorganizado, difícil de entender y sin modularidad.	Código parcialmente organizado con escasa separación entre lógica e interfaz gráfica.	Código organizado y modular, separando adecuadamente funciones, clases e interfaz.	Código limpio, reutilizable y bien documentado, aplicando buenas prácticas de programación orientada a objetos.
Interfaz Gráfica de Usuario (10%)	No presenta interfaz gráfica o la interfaz es inutilizable.	Interfaz básica con problemas de navegación, validación o visualización de resultados.	Interfaz funcional y clara, con validación adecuada de entradas y resultados visibles.	Interfaz intuitiva, amigable y visualmente organizada, mostrando claramente conversiones, evaluaciones y mensajes de error.
Pruebas y Validaciones (5%)	No presenta pruebas ni evidencias de funcionamiento.	Presenta pocas pruebas y con escasa documentación.	Realiza pruebas suficientes incluyendo expresiones válidas e inválidas.	Presenta pruebas exhaustivas y documentadas, incluyendo expresiones complejas, manejo de errores y validaciones completas.